

1. Your project: HoWiE chatbot

What the project wants:

HoWiE is a RAG chatbot for exploring higher-education and science-research results. The goal is not just answering questions, but helping users extract, combine, and contextualize findings from heterogeneous documents like papers, reports, policy briefs, tables, and portal content. The project also explicitly asks about different question types: summarizing, comparing, interpreting, and navigating.

How this affects your chatbot:

Your chatbot should not treat every question the same way. It should know whether the user is asking for:

- a fact,
- a summary,
- a comparison,
- a methodology explanation,
- a policy interpretation,
- or source navigation.

This is already close to what you built with question-type detection.

2. Lewis et al. 2020 — Original RAG paper

What they researched:

This is the foundational RAG paper. It combines a **retriever** and a **generator**. The retriever finds relevant documents; the generator uses those documents to produce an answer. RAG was designed to solve the problem that LLMs store knowledge in their parameters but cannot easily update it, verify it, or cite its source.

How to implement it in your chatbot:

This is basically your current system:

```
User question
→ embed question
→ retrieve top-k chunks
→ send chunks + question to local LLM
→ generate answer
```

What you learn from it:

This is your **Baseline RAG**. In your thesis, you can say: "First, I implemented a standard RAG pipeline inspired by Lewis et al."

3. Gao et al. / RAG survey papers

What they researched:

The RAG surveys divide RAG into **Naive RAG**, **Advanced RAG**, and **Modular RAG**. They explain that simple RAG is not enough because retrieval can be noisy, context may be irrelevant, and the generator may hallucinate. They also describe modules like query rewriting, reranking, hybrid search, compression, iterative retrieval, and adaptive retrieval.

How to implement it in your chatbot:

Use this as your literature-review backbone. Your system can evolve like this:

Naive RAG:

PDF → chunks → embeddings → top-k retrieval → answer

Advanced RAG:

PDF → metadata-aware chunks → hybrid retrieval → reranking → answer

Modular RAG:

intent classifier → choose retrieval strategy → retrieve → verify → answer

What you learn from it:

Your contribution should not be "I built RAG." Your contribution should be "I improved Naive RAG for HoWiE using question-type-aware retrieval and evaluation."

4. Shuster et al. 2021 — Retrieval reduces hallucination in conversation

What they researched:

They studied RAG for dialogue systems and showed that retrieval can reduce hallucination in chatbots. They used retrievers, rankers, and encoder-decoder models to improve knowledge-grounded conversation. Their best models reduced hallucinated responses by more than 60% in human evaluation.

How to implement it in your chatbot:

Your chatbot should keep conversation history, not just the latest question.

Example:

Previous user questions

+ current question

→ rewrite into standalone query

→ retrieve relevant chunks

→ generate grounded answer

What you learn from it:

For HoWiE, users may ask follow-up questions like:

"What about international students?"

"Compare that with the previous report."

"Which source said that?"

So you need **conversation-aware retrieval**, not only single-question retrieval.

5. Swacha & Gracel 2025 — RAG chatbots for education survey

What they researched:

They reviewed 47 RAG chatbot papers in education. They found that education RAG chatbots are used for learning support, institutional knowledge, course guidance, and subject-specific databases. They also found that evaluation is very diverse: accuracy, relevance, faithfulness, generated text quality, usability, and user satisfaction are all used.

How to implement it in your chatbot:

Use this paper to justify why your domain is valid. HoWiE is similar to education/science-communication chatbots, but your focus is higher-education research transfer.

What you learn from it:

Your evaluation should not only check answer accuracy. You should evaluate:

- retrieval quality,
- answer relevance,
- faithfulness,
- citation quality,
- user usefulness,
- clarity.

6. RAPTOR — Tree-based retrieval

What they researched:

RAPTOR solves a weakness of normal chunk retrieval. Normal RAG retrieves small chunks, but some questions need a higher-level understanding of the whole document. RAPTOR builds a tree: small chunks at the bottom, summaries of related chunks above them, and higher-level summaries above those. It retrieves from different levels depending on the question.

How to implement it in your chatbot:

For each PDF:

```
PDF
→ split into small chunks
→ embed chunks
→ cluster similar chunks
→ summarize each cluster
```

→ embed summaries

At query time:

Fact question → retrieve small chunks
Summary question → retrieve section summaries
Broad question → retrieve high-level summaries

Why this is useful for HoWiE:

If someone asks:

"What are the main findings of this study?"

Small chunks may not be enough. RAPTOR-style summaries can help.

7. GraphRAG — Graph-based retrieval

What they researched:

GraphRAG is designed for **global questions** over a whole corpus. Normal vector RAG works when the answer is in a few chunks. But if the question is "What are the main themes across all documents?", vector RAG fails. GraphRAG extracts entities, relationships, and claims, builds a knowledge graph, detects communities, summarizes them, and answers global questions using map-reduce summarization.

How to implement it in your chatbot:

You do not need full GraphRAG immediately. A simplified version is enough:

Extract entities:

- universities
- students
- countries
- policies
- indicators
- research topics

Extract relationships:

- study X reports finding Y
- policy Z affects group A
- dataset B measures indicator C

Build graph:

entity → relationship → entity

Then for broad questions:

"What themes appear across the reports?"
→ retrieve graph communities / topic summaries
→ generate answer

Why this is useful for HoWiE:

HoWiE users may ask corpus-level questions like:

"What are the recurring findings about student mobility?"
"Which topics are most common across DZHW reports?"
"How do different studies talk about internationalization?"

That is GraphRAG territory.

8. Self-RAG — Retrieve, generate, critique

What they researched:

Self-RAG teaches the model to decide when retrieval is needed, judge whether retrieved passages are relevant, generate the answer, and critique whether the answer is supported. It uses reflection tokens to control retrieval and self-evaluation.

How to implement it in your chatbot:

You do not need to train a full Self-RAG model. You can implement a simpler pipeline:

```
Step 1: Does this question need retrieval?  
Step 2: Are retrieved chunks relevant?  
Step 3: Is the answer supported by the chunks?  
Step 4: If not supported, say "I cannot answer from the provided documents."
```

Example logic:

```
if intent in ["fact", "comparison", "methodology", "policy"]:  
    use_retrieval = True  
else:  
    use_retrieval = maybe
```

Why this is useful for your thesis:

It gives you a research direction: **adaptive retrieval and self-checking for HoWiE.**

9. Adaptive RAG / RAGate

What they researched:

This paper asks: "Should RAG be used for every conversation turn?" Their answer is no. Sometimes retrieval adds irrelevant information and makes the response worse. They build RAGate, a gate that decides whether a conversational response needs external knowledge.

How to implement it in your chatbot:

Add a small decision layer before retrieval:

```
User question
→ classify:
  needs document evidence?
  general explanation?
  follow-up clarification?
  source navigation?
→ if needed, retrieve
→ if not needed, answer normally or ask clarification
```

Example:

User: "What is RAG?"

Maybe no document retrieval needed.

User: "What does the DZHW report say about student dropout?"

Retrieval needed.

Why this matters:

This makes your chatbot more natural and efficient.

10. CRAG — Corrective RAG

What they researched:

CRAG focuses on what happens when retrieval goes wrong. It adds a lightweight retrieval evaluator that decides whether retrieved documents are correct, incorrect, or ambiguous. If retrieval is bad, the system corrects or supplements the retrieval. It also decomposes retrieved documents into smaller knowledge strips and filters irrelevant parts.

How to implement it in your chatbot:

After retrieval, add a relevance-checking stage:

```
Retrieved chunks
→ score each chunk for relevance
→ remove weak chunks
→ if all chunks are weak:
  say insufficient evidence
else:
  generate answer
```

For local HoWiE, you probably should not use web search at first. Instead, use:

```
low confidence → retrieve again with rewritten query
low confidence again → refuse / ask user to choose document
```

Why this is a very good thesis module:

It directly improves your current chatbot. Your current version probably retrieves something even when the answer is not really there. CRAG gives you a way to detect that.

11. RAGAS — Automatic RAG evaluation

What they researched:

RAGAS evaluates RAG without needing ground-truth answers for everything. It measures context relevance, answer relevance, and answer faithfulness. It checks whether the answer statements are supported by the retrieved context.

How to implement it in your chatbot:

For each test question, store:

```
question
retrieved chunks
generated answer
source citations
```

Then evaluate:

```
context relevance: were the retrieved chunks useful?
answer relevance: did the answer address the question?
faithfulness: is the answer supported by the chunks?
```

Why this matters:

This gives your thesis measurable results.

12. ARES — Better RAG evaluation with synthetic data

What they researched:

ARES creates synthetic question-answer data from your corpus, trains lightweight judge models, and evaluates RAG systems on context relevance, answer faithfulness, and answer relevance. It also uses a small human validation set to improve reliability.

How to implement it in your chatbot:

You can do a simplified ARES-inspired evaluation:

```
Take HoWiE documents
→ generate synthetic questions from chunks
→ create correct and incorrect answers
→ evaluate your chatbot
→ manually label 50-100 examples
```

Why this is useful:

You do not need thousands of manually labelled questions. You can generate a first evaluation dataset semi-automatically.

13. RGB benchmark — Testing RAG weaknesses

What they researched:

RGB evaluates four abilities of RAG systems: noise robustness, negative rejection, information integration, and counterfactual robustness. It found that models still struggle with refusing unsupported questions, integrating multiple documents, and handling false retrieved information.

How to implement it in your chatbot:

Create a HoWiE test set with four categories:

1. Noise robustness:
relevant + irrelevant chunks mixed
2. Negative rejection:
no chunk contains the answer
3. Information integration:
answer requires multiple reports
4. Counterfactual robustness:
retrieved chunk contains misleading/contradictory info

Why this is excellent for your thesis:

This gives you strong evaluation tasks. You can show whether your improved RAG is better than your baseline RAG.

14. Correctness is not Faithfulness — Citation faithfulness

What they researched:

This paper says citations can be correct but still not faithful. A model may already know an answer, then attach a citation afterward, even if it did not actually use the cited document. They introduce **citation faithfulness**, meaning the cited document must both support the claim and causally influence the answer. They found post-rationalized citations can be a serious problem.

How to implement it in your chatbot:

Your chatbot should not just output citations at the end. It should attach evidence sentence by sentence.

Better answer format:

Answer:
Germany had X students in year Y. [Source: Report A, p. 12]

Evidence:
The source states: "...exact sentence..."

Then run a check:

For each generated claim:
→ find supporting chunk
→ if no supporting chunk, remove claim or mark as interpretation

Why this is very important for HoWiE:

HoWiE is about research transfer. Wrong or decorative citations would be dangerous. This paper gives you a strong research gap.

15. Sci2Pol — Science-to-policy brief generation

What they researched:

Sci2Pol studies how LLMs can turn scientific papers into policy briefs. It breaks the task into stages: understanding, summarization, generation, and verification. It also shows that normal text metrics like ROUGE and BERTScore may not properly capture policy-brief quality.

How to implement it in your chatbot:

For HoWiE, some users may be policymakers or university managers. Your chatbot can produce answers in different styles:

For researcher → detailed evidence answer
For policy user → short policy-oriented summary
For journalist → plain-language explanation
For manager → actionable bullet points

Why this matters:

It connects directly to HoWiE's stakeholder idea.

16. Kessler et al. 2025 — AI in science communication

What they researched:

This paper discusses how AI changes science communication. It says AI can simplify complex science, support interaction, and personalize communication, but also brings risks: hallucination, bias, lack of transparency, and overtrust.

How to implement it in your chatbot:

Your chatbot should be designed as a science-communication tool, not just a technical QA bot.

So add:

- plain-language mode
- evidence mode



- source-first mode
- uncertainty statements
- “what this does not show” section

Why this matters:

It helps you justify the social and communication relevance of your thesis.

17. ALES academic chatbot paper

What they researched:

ALES is an academic LLM chatbot for higher education. It combines RAG, citation-aware output, and ethical safeguards. The paper identifies gaps in academic AI systems: lack of transparency, weak citation verifiability, poor institutional integration, and limited ethical safeguards.

How to implement it in your chatbot:

For HoWiE, add:

- source citations
- document-level traceability
- answer limitations
- no unsupported claims
- institutional/document metadata

Why this is useful:

It is very close to your domain: academic chatbot + RAG + source trust.

The implementation you should build now

Based on all these papers, your improved chatbot should look like this:

1. User asks a question
2. Intent classifier
 - fact
 - summary
 - comparison
 - methodology
 - interpretation
 - navigation
 - general chat
3. Retrieval gate
 - decide whether retrieval is needed
4. Query rewriting
 - turn follow-up question into standalone query
5. Retrieval strategy based on intent

- fact → small chunks
- summary → section summaries / RAPTOR
- comparison → multi-document retrieval
- methodology → method-section metadata
- global theme → GraphRAG-style summaries
- navigation → document/section search

6. Retrieval evaluator

- remove irrelevant chunks
- detect insufficient evidence

7. Answer generation

- answer only from retrieved evidence

8. Citation faithfulness check

- every claim must map to a source chunk

9. Final answer

- evidence
- summary
- interpretation
- citations
- limitations

Best thesis title from these papers

A strong title would be:

Question-Type-Aware and Evidence-Faithful Retrieval-Augmented Generation for Exploring Higher-Education Research Documents

Your research contribution would be:

I started with a standard local RAG chatbot and improved it using question-type-aware retrieval, adaptive retrieval decisions, retrieval quality checking, and citation-faithfulness evaluation for the HoWiE domain.

That is a very defensible Master's thesis.